

Lab 11

Course: Programming Design and
Implementation

Lecturer: Bou Channa

Name: Kiv Sovannlyda

ID: p20240001



EXERCISE 1

Given an array `int array[] = {90, 6, -10, 2, 70, 9, 1, 5, 22};`

Write a Java program to sort this array in an ascending order using the Bubble sort algorithm. The pseudo code for bubble sort is provided below.

Note: To get size of array: `array.length`

Array: 90, 6, -10, 2, 70, 9, 1, 5, 22

⇒ Sort in ascending order:
-10 1 2 5 6 9 22 70 90

```
BubbleSort(array, n)
  for i from 0 to n-1
    for j from 0 to n-i-2
      if array[j] > array[j + 1]
        Swap(array[j], array[j + 1])
      end if
    end for
  end for
end BubbleSort
```

```
J ex01.java > ex01 > main(String[])
1 public class ex01 {
2     public static void bubbleSort(int[] arr) {
3         int n = arr.length;
4         for (int i = 0; i < n - 1; i++) {
5             for (int j = 0; j < n - i - 1; j++) {
6                 if (arr[j] > arr[j + 1]) {
7                     // Swap
8                     int temp = arr[j];
9                     arr[j] = arr[j + 1];
10                    arr[j + 1] = temp;
11                }
12            }
13        }
14    }
15
16    Run | Debug
17    public static void main(String[] args) {
18        int[] array = {90, 6, -10, 2, 70, 9, 1, 5, 22};
19        bubbleSort(array);
20        System.out.print(s: "Sorted: ");
21        for (int num : array) {
22            System.out.print(num + " ");
23        }
24        System.out.println();
25    }
26 }
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS

```
● PS C:\Users\Admin\Documents\y2\PDI\TP\TP11> & 'C:\Program Files\J
ptionMessages' '-cp' 'C:\Users\Admin\AppData\Roaming\Code\User\wor
92813ed5\bin' 'ex01'
Sorted: -10 1 2 5 6 9 22 70 90
○ PS C:\Users\Admin\Documents\y2\PDI\TP\TP11> █
```

EXERCISE 2

Ask user for 10 numbers. Then sort these numbers in an descending order.

Write a program in Java to sort this array in an descending order using the Selection sort algorithm.

Input: 2 40 8 20 -1 0 60 40 5 9

⇒ Sort in descending order:

60 40 40 20 9 8 5 2 -1 0

```
1  import java.util.*;
2  public class ex02 {
3      public static void main(String[] args) {
4          Scanner scanner = new Scanner(System.in);
5          int[] array = new int[10];
6
7          System.out.println("Enter 10 numbers:");
8          for (int i = 0; i < 10; i++) {
9              System.out.print("Number " + (i + 1) + ": ");
10             array[i] = scanner.nextInt();
11         }
12
13         selectionSort(array);
14
15         System.out.print("Sorted: ");
16         for (int n : array) {
17             System.out.print(n + " ");
18         }
19         System.out.println();
20         scanner.close();
21     }
22
23     public static void selectionSort(int[] array) {
24         int n = array.length;
25         for (int i = 0; i < n - 1; i++) {
26             int maxIndex = i;
27             for (int j = i + 1; j < n; j++) {
28                 if (array[j] > array[maxIndex]) {
29                     maxIndex = j;
30                 }
31             }
32
33             int temp = array[i];
34             array[i] = array[maxIndex];
35             array[maxIndex] = temp;
36         }
37     }
38 }
```

```
● Enter 10 numbers:
Number 1: 10
Number 2: 20
Number 3: -90
Number 4: 100
Number 5: -800
Number 6: 67
Number 7: 12
Number 8: 1
Number 9: -20
Number 10: 4
Sorted: 100 67 20 12 10 4 1 -20 -90 -800
```

EXERCISE 3

Given an array `String name[] = {"Dara", "Sok", "Sokha", "Daro", "Panha", "Pisey"};`

Write a program in Java to sort this array in an ascending order using the Selection sort algorithm. The pseudo code for selection sort is provided below.

Hint: You can not compare strings with comparison operator. You can use string method below.

`String.compareTo()` method is used to compare two strings lexicographically (based on Unicode values).

`String.compareToIgnoreCase()`

```
int result1 = string1.compareTo(string2);
int result2 = string1.compareToIgnoreCase(string2);
```

Return from this function can be:

- 0 : when both strings are the same
- Negative : when string1 is less than string2
- Positive : when string1 is bigger than string2

```
SelectionSort(array, n)
for i from 0 to n-1
    minIndex = i
    for j from i+1 to n-1
        if array[j] < array[minIndex]
            minIndex = j
    end for
    Swap(array[i], array[minIndex])
end for
end SelectionSort
```

```
J ex03.java > ...
1 public class ex03 {
2     public static void sortSelection(String name[]){
3         for (int i = 0; i < name.length - 1; i++) {
4             int minIndex = i;
5
6             for (int j = i + 1; j < name.length; j++) {
7                 if (name[j].compareToIgnoreCase(name[minIndex]) < 0) {
8                     minIndex = j;
9                 }
10            }
11            swap(name, i, minIndex);
12        }
13    }
14    static void swap (String[] arr, int i, int j) {
15        String temp = arr[i];
16        arr[i] = arr[j];
17        arr[j] = temp;
18    }
19    public static void main(String[] args) {
20        String name[] = {"Dara", "Sok", "Sokha", "Daro", "Panha", "Pisey"};
21        sortSelection(name);
22        for (String n : name) {
23            System.out.println(n);
24        }
25    }
26 }
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS

```
PS C:\Users\Admin\Documents\y2\PDI\TP\TP11> & 'C:\Program Files\Java\jdk-25\bin\java.exe
ssages' '-cp' 'C:\Users\Admin\AppData\Roaming\Code\User\workspaceStorage\f66e9a7cff771e8
'ex03'
```

```
● Dara
Daro
Panha
Pisey
Sok
Sokha
```

EXERCISE 4

```
int[] array = new int[10];
Random random = new Random();
// Generates a random number between 0 and 99
for (int i = 0; i < array.length; i++) {
    array[i] = random.nextInt(100);
}
```

An example to generate 10 numbers randomly

Let's create an array containing 25 numbers. These numbers are randomly generated by the computer.

Write a program in Java to sort this array using the Insertion sort algorithm. The pseudo code for insertion sort is provided below.

- A. Sort with descending order
- B. Sort with ascending order

```
InsertionSort(array, n)
for i from 1 to n-1
    key = array[i]
    j = i - 1
    while j >= 0 and array[j] > key
        array[j + 1] = array[j]
        j = j - 1
    end while
    array[j + 1] = key
end for
end InsertionSort
```

```
1  import java.util.Random;
2
3  public class ex04 {
4      public static void main(String[] args) {
5          int[] array = new int[25];
6          Random random = new Random();
7
8          for (int i = 0; i < array.length; i++) {
9              array[i] = random.nextInt(100);
10         }
11
12         int[] descArray = array.clone();
13         insertionSortDescending(descArray);
14         System.out.print("Descending: ");
15         printArray(descArray);
16
17         int[] ascArray = array.clone();
18         insertionSortAscending(ascArray);
19         System.out.print("Ascending: ");
20         printArray(ascArray);
21         System.out.println();
22     }
23
24     public static void printArray(int[] arr) {
25         for (int i = 0; i < arr.length; i++) {
26             System.out.print(arr[i]);
27             if (i < arr.length - 1) System.out.print(", ");
28         }
29         System.out.println();
30     }
31
32     public static void insertionSortAscending(int[] array) {
33         int n = array.length;
34         for (int i = 1; i < n; i++) {
35             int num = array[i];
36             int j = i - 1;
37             while (j >= 0 && array[j] > num) {
38                 array[j + 1] = array[j];
39                 j = j - 1;
40             }
41             array[j + 1] = num;
42         }
43     }
44
45     public static void insertionSortDescending(int[] array) {
46         int n = array.length;
47         for (int i = 1; i < n; i++) {
48             int num = array[i];
49             int j = i - 1;
50             while (j >= 0 && array[j] < num) {
51                 array[j + 1] = array[j];
52                 j = j - 1;
53             }
54             array[j + 1] = num;
55         }
56     }
57 }
58
59
```

```
PS C:\Users\Admin\Documents\y2\PDI\TP\TP11> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '--enable-preview' '
ptionMessages' '-cp' 'C:\Users\Admin\AppData\Roaming\Code\User\workspaceStorage\f66e9a7cff771e8ed2046a348911a2b
92813ed5\bin' 'ex04'
Descending: 92, 82, 76, 75, 75, 73, 68, 67, 60, 57, 54, 53, 49, 49, 48, 46, 33, 31, 29, 26, 23, 23, 22, 20, 9
Ascending: 9, 20, 22, 23, 23, 26, 29, 31, 33, 46, 48, 49, 49, 53, 54, 57, 60, 67, 68, 73, 75, 75, 76, 82, 92
```

EXERCISE 5

Do a case study about merge sorting algorithm.

Given an array below. Let's sort this data using merge sort.

float scores[] = {90.5, 20, 50, 75, 80, 85, 69, 0, 80, 40, 70};

```
1 public class ex05 {
2     public static void main(String[] args) {
3         float[] scores = {90.5f, 20, 50, 75, 80, 85, 69, 0, 80, 40, 70};
4         mergeSort(scores, 0, scores.length - 1);
5
6         System.out.print("Sorted: ");
7         printFloatArray(scores);
8         System.out.println();
9     }
10
11     public static void printFloatArray(float[] array) {
12         for (int i = 0; i < array.length; i++) {
13             System.out.print(array[i]);
14             if (i < array.length - 1) {
15                 System.out.print(", ");
16             }
17         }
18     }
19     public static void mergeSort(float[] array, int left, int right) {
20         if (left < right) {
21             int mid = (left + right) / 2;
22             mergeSort(array, left, mid);
23             mergeSort(array, mid + 1, right);
24             merge(array, left, mid, right);
25         }
26     }
27
28     public static void merge(float[] array, int left, int mid, int right) {
29         int n1 = mid - left + 1;
30         int n2 = right - mid;
31
32         float[] leftArray = new float[n1];
33         float[] rightArray = new float[n2];
34
35         for (int i = 0; i < n1; i++)
36             leftArray[i] = array[left + i];
37         for (int j = 0; j < n2; j++)
38             rightArray[j] = array[mid + 1 + j];
39
40         int i = 0, j = 0, k = left;
41         while (i < n1 && j < n2) {
42             if (leftArray[i] <= rightArray[j]) {
43                 array[k] = leftArray[i];
44                 i++;
45             } else {
46                 array[k] = rightArray[j];
47                 j++;
48             }
49             k++;
50         }
51
52         while (i < n1) {
53             array[k] = leftArray[i];
54             i++;
55             k++;
56         }
57
58         while (j < n2) {
59             array[k] = rightArray[j];
60             j++;
61             k++;
62         }
63     }
64 }
65
```

Output:

```
● Sorted:  0.0, 20.0, 40.0, 50.0, 69.0, 70.0, 75.0, 80.0, 80.0, 85.0, 90.5
```

EXERCISE 6

Create a class Car (vin, model, year, price, engineType).
Initialize 10 data in a list.

Use one of the following sorting methods to order this product list
by price (descending order). Then display data on screen.

1. Selection sort
2. Insertion sort
3. Bubble sort
4. Merge sort

```
1  import java.util.ArrayList;
2
3  public class ex06 {
4      static class Car {
5          String vin;
6          String model;
7          int year;
8          double price;
9          String engineType;
10
11         public Car(String vin, String model, int year, double price, String engineType) {
12             this.vin = vin;
13             this.model = model;
14             this.year = year;
15             this.price = price;
16             this.engineType = engineType;
17         }
18
19         @Override
20         public String toString() {
21             return String.format("%-15s %-15s %d $%-10.2f %s",
22                 vin, model, year, price, engineType);
23         }
24     }
25
26     public static void main(String[] args) {
27         selectionSort();
28     }
29
30     public static void selectionSort() {
31         ArrayList<Car> cars = new ArrayList<>();
32         cars.add(new Car("VIN004", "Ford Mustang", 2021, 35000, "Gas"));
33         cars.add(new Car("VIN002", "Audi A4", 2022, 40000, "Gas"));
34         cars.add(new Car("VIN0012", "Nissan Altima", 2020, 24000, "Gas"));
35         cars.add(new Car("VIN010", "Lexus ES", 2023, 48000, "Hybrid"));
36
37         System.out.println("\nBefore Sorting:");
38         System.out.println("VIN          Model          Year  Price      Engine");
39         System.out.println("-----");
40         for (Car car : cars) {
41             System.out.println(car);
42         }
43
44         selectionSortCars(cars);
45
46         System.out.println("\nAfter Sorting (by Price - Descending):");
47         System.out.println("VIN          Model          Year  Price      Engine");
48         System.out.println("-----");
49         for (Car car : cars) {
50             System.out.println(car);
51         }
52         System.out.println();
53     }
54
55     public static void selectionSortCars(ArrayList<Car> cars) {
56         int n = cars.size();
57         for (int i = 0; i < n - 1; i++) {
58             int maxIndex = i;
59             for (int j = i + 1; j < n; j++) {
60                 if (cars.get(j).price > cars.get(maxIndex).price) {
61                     maxIndex = j;
62                 }
63             }
64             Car temp = cars.get(i);
65             cars.set(i, cars.get(maxIndex));
66             cars.set(maxIndex, temp);
67         }
68     }
69 }
70
```


Output:

Before Sorting:				
VIN	Model	Year	Price	Engine
VIN004	Ford Mustang	2021	\$35000.00	Gas
VIN002	Audi A4	2022	\$40000.00	Gas
VIN0012	Nissan Altima	2020	\$24000.00	Gas
VIN010	Lexus ES	2023	\$48000.00	Hybrid
After Sorting (by Price - Descending):				
VIN	Model	Year	Price	Engine
VIN010	Lexus ES	2023	\$48000.00	Hybrid
VIN002	Audi A4	2022	\$40000.00	Gas
VIN004	Ford Mustang	2021	\$35000.00	Gas
VIN0012	Nissan Altima	2020	\$24000.00	Gas

EXERCISE 7

Read this CSV data.

Sort this data based on average score. Finally write them to a new CSV file.

Using either one of the following sort methods: Bubble, Selection, Insertion, Quick, Merge sort.

#	A	B	C	D	E	F	G	H	I
1	List of candidates with the test scores								
2	ID	gender	group	Education	Math	English	Computer	Total	Average
3	760	male	group A	associate degree	47	57	44	148	49.33
4	3391	male	group C	some college	76	78	75	229	76.33
5	3841	male	group B	some college	40	43	39	122	40.67
6	2632	male	group D	high school	64	64	67	195	65
7	2779	male	group C	associate degree	58	54	52	164	54.67
8	1608	male	group D	associate degree	40	52	43	135	45
9	1354	male	group A	some college	78	72	70	220	73.33
10	4517	male	group C	high school	88	89	86	263	87.67
11	9332	male	group C	master degree	46	42	46	134	44.67
12	3306	male	group D	high school	66	69	63	198	66
13	8979	male	group D	some college	44	54	53	151	50.33
14	7703	male	group D	bachelor degree	74	71	80	225	75
15	7613	male	group A	master degree	73	74	72	219	73
16	1621	male	group B	some college	69	54	55	178	59.33
17	7887	male	group C	high school	70	70	65	205	68.33
18	7964	male	group D	some college	40	42	38	120	40
19	8324	male	group E	some college	97	87	82	266	88.67
20	8675	male	group E	associate degree	81	81	79	241	80.33
21	8981	male	group B	associate degree	57	56	57	170	56.67
22	7450	male	group C	associate degree	55	61	54	170	56.67
23	1558	male	group B	some college	59	65	66	190	63.33
24	2418	male	group B	associate degree	65	54	57	176	58.67
25	646	male	group C	high school	82	84	82	248	82.67
26	6700	male	group E	some college	53	55	48	156	52
27	287	male	group E	associate degree	77	69	68	214	71.33

Download dataset here:
<https://shorturl.at/Q6K4O>

```

1  import java.io.*;
2  import java.util.*;
3
4  public class ex07 {
5
6      static class Candidate {
7          String id;
8          String gender;
9          String group;
10         String education;
11         String testPreparationCourse;
12         int math;
13         int english;
14         int computer;
15         int total;
16         double average;
17
18         public Candidate(String line) {
19             String[] parts = line.split(",");
20             this.id = parts[0];
21             this.gender = parts[1];
22             this.group = parts[2];
23             this.education = parts[3];
24             this.testPreparationCourse = parts[4];
25             this.math = Integer.parseInt(parts[5]);
26             this.english = Integer.parseInt(parts[6]);
27             this.computer = Integer.parseInt(parts[7]);
28             this.total = Integer.parseInt(parts[8]);
29             this.average = Double.parseDouble(parts[9]);
30         }
31
32         public String toCSV() {
33             return id + "," + gender + "," + group + "," + education + "," +
34                 testPreparationCourse + "," + math + "," + english + "," + computer + "," + total + "," + average;
35         }
36     }
37
38     public static void bubbleSort(Candidate[] candidates) {
39         int n = candidates.length;
40         for (int i = 0; i < n - 1; i++) {
41             for (int j = 0; j < n - i - 1; j++) {
42                 if (candidates[j].average > candidates[j + 1].average) {
43                     // Swap candidates[j] and candidates[j+1]
44                     Candidate temp = candidates[j];
45                     candidates[j] = candidates[j + 1];
46                     candidates[j + 1] = temp;
47                 }
48             }
49         }
50     }
51
52     public static void main(String[] args) {
53         try {
54             BufferedReader reader = new BufferedReader(new FileReader("C:/Users/Admin/Documents/y2/PDI/TP/TP11/CandidatePerformanceTest.csv"));
55             reader.readLine();
56             reader.readLine();
57
58             List<Candidate> candidateList = new ArrayList<>();
59             String line;
60             int lineNum = 1;
61             while ((line = reader.readLine()) != null) {
62                 if (line.trim().isEmpty()) continue;
63                 System.out.println("Reading line " + lineNum + ": " + line);
64                 candidateList.add(new Candidate(line));
65                 lineNum++;
66             }
67             reader.close();
68
69             Candidate[] candidates = candidateList.toArray(new Candidate[0]);
70
71             bubbleSort(candidates);
72
73             BufferedWriter writer = new BufferedWriter(new FileWriter("sorted_candidates.csv"));
74             writer.write("ID,gender,group,Education,test_preparation_course,Math,English,Computer,Total,Average\n");
75
76             for (Candidate candidate : candidates) {
77                 writer.write(candidate.toCSV() + "\n");
78             }
79             writer.close();
80
81             System.out.println("Data sorted by average score using Bubble Sort!");
82             System.out.println("Sorted data written to 'sorted_candidates.csv'");
83
84         } catch (IOException e) {
85             System.out.println("Error: " + e.getMessage());
86             e.printStackTrace();
87         }
88     }
89 }

```

Original CSV:

List of candidates with the test scores										
ID	gender	group	Education	test_prep	Math	English	Computer	Total	Average	
760	male	group A	associate	none	47	57	44	148	49.33	
3391	male	group C	some colle	none	76	78	75	229	76.33	
3841	male	group B	some colle	none	40	43	39	122	40.67	
2632	male	group D	high schoc	completed	64	64	67	195	65	
2779	male	group C	associate	none	58	54	52	164	54.67	
1608	male	group D	associate	none	40	52	43	135	45	
1354	male	group A	some colle	completed	78	72	70	220	73.33	
4517	male	group C	high schoc	none	88	89	86	263	87.67	
9332	male	group C	master deg	completed	46	42	46	134	44.67	
3306	male	group D	high schoc	none	66	69	63	198	66	
8979	male	group D	some colle	none	44	54	53	151	50.33	
7703	male	group D	bachelor d	completed	74	71	80	225	75	
7613	male	group A	master deg	none	73	74	72	219	73	
1621	male	group B	some colle	none	69	54	55	178	59.33	
7887	male	group C	high schoc	none	70	70	65	205	68.33	
7964	male	group D	some colle	none	40	42	38	120	40	
8324	male	group E	some colle	none	97	87	82	266	88.67	
8675	male	group E	associate	completed	81	81	79	241	80.33	
8981	male	group B	associate	none	57	56	57	170	56.67	
7450	male	group C	associate	none	55	61	54	170	56.67	
1558	male	group B	some colle	completed	59	65	66	190	63.33	
2418	male	group B	associate	none	65	54	57	176	58.67	
646	male	group C	high schoc	completed	82	84	82	248	82.67	
6700	male	group E	some colle	none	53	55	48	156	52	
287	male	group E	associate	completed	77	69	68	214	71.33	
4242	male	group C	some colle	none	53	44	42	139	46.33	
8557	male	group D	high schoc	none	88	78	75	241	80.33	
6123	male	group D	associate	none	52	55	49	156	52	
5500	male	group D	some colle	completed	58	59	58	175	58.33	
2519	male	group E	bachelor d	completed	79	74	72	225	75	
8654	male	group A	some high	none	39	39	34	112	37.33	
1883	male	group A	associate	none	62	61	55	178	59.33	
7136	male	group B	some high	none	67	64	61	192	64	
7938	male	group D	some high	none	45	37	37	119	39.67	
5292	male	group B	associate	none	61	58	56	175	58.33	
8219	male	group D	some colle	completed	63	55	63	181	60.33	
9189	male	group C	some high	none	61	57	56	174	58	
3172	male	group C	some high	none	49	49	41	139	46.33	
4540	male	group B	associate	none	44	41	38	123	41	
3067	male	group E	some high	none	30	26	22	78	26	

Output after running code:

ID	gender	group	Education	test_prep	Math	English	Computer	Total	Average
2620	female	group C	some high	none	0	17	10	27	9
5947	male	group B	high schoc	none	30	24	15	69	23
7084	male	group A	some colle	none	28	23	19	70	23.33
3067	male	group E	some high	none	30	26	22	78	26
4028	female	group B	some high	none	18	32	28	78	26
182	female	group C	high schoc	none	29	29	30	88	29.33
3611	female	group B	some high	none	24	38	27	89	29.67
1700	female	group B	some colle	none	19	38	32	89	29.67
1572	male	group C	some colle	none	35	28	27	90	30
3435	male	group B	high schoc	none	36	29	27	92	30.67
4415	female	group D	some high	none	27	34	32	93	31
1543	female	group C	some colle	none	22	39	33	94	31.33
1765	female	group D	associate	none	26	31	38	95	31.67
673	male	group C	high schoc	none	27	34	36	97	32.33
9584	female	group B	high schoc	completed	23	44	36	103	34.33
8370	female	group C	some colle	none	32	39	33	104	34.67
2414	female	group E	some high	none	32	34	38	104	34.67
8654	male	group A	some high	none	39	39	34	112	37.33
3465	female	group C	some high	completed	29	40	44	113	37.67
8896	male	group B	some colle	none	41	39	34	114	38
5938	male	group D	high schoc	none	42	39	34	115	38.33
1148	female	group C	high schoc	none	34	42	39	115	38.33
3603	male	group D	high schoc	none	46	34	36	116	38.67
5757	female	group C	high schoc	none	33	41	43	117	39
4820	female	group D	bachelor d	none	29	41	47	117	39
8846	male	group A	some high	none	51	31	36	118	39.33
7938	male	group D	some high	none	45	37	37	119	39.67
1304	male	group C	associate	none	47	37	35	119	39.67
3499	male	group D	bachelor d	completed	39	42	38	119	39.67
7964	male	group D	some colle	none	40	42	38	120	40
1040	female	group E	bachelor d	none	37	45	38	120	40
3841	male	group B	some colle	none	40	43	39	122	40.67
5531	female	group C	some colle	none	35	44	43	122	40.67
4540	male	group B	associate	none	44	41	38	123	41
5093	female	group A	high schoc	completed	34	48	41	123	41
9148	female	group A	some high	none	38	43	43	124	41.33
9666	female	group E	high schoc	none	41	45	40	126	42
8159	female	group B	some high	completed	32	51	44	127	42.33
4670	male	group C	some colle	none	53	39	37	129	43
7493	female	group B	some high	none	37	46	46	129	43
1014	male	group C	some high	completed	53	37	40	130	43.33